

Infix to Postfix conversion and the Evaluation

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <math.h>

#define MAX 100

// Stacks
char stack[MAX];
int top = -1;

double evalStack[MAX];
int evalTop = -1;

// ----- Stack for Operators -----
void push(char c)
{
    if (top == MAX - 1)
    {
        printf("Stack overflow!\n");
        return;
    }
}
```

```

        stack[++top] = c;
    }

char pop()
{
    if (top == -1)
        return '\0';
    return stack[top--];
}

char peek()
{
    if (top == -1)
        return '\0';
    return stack[top];
}

// ----- Stack for Evaluation -----
void pushEval(double val)
{
    evalStack[++evalTop] = val;
}

double popEval()
{
    return evalStack[evalTop--];
}

```

```

// ----- Helper Functions -----

int precedence(char op)
{
    if (op == '^')
        return 3;
    if (op == '*' || op == '/')
        return 2;
    if (op == '+' || op == '-')
        return 1;
    return 0;
}

int isOperator(char c)
{
    return (c == '+' || c == '-' || c == '*' || c == '/' ||
c == '^');
}

// ----- Infix → Postfix -----

void infixToPostfix(char infix[], char postfix[])
{
    int i, k = 0;
    char ch;

    for (i = 0; infix[i] != '\0'; i++)
    {
        ch = infix[i];

```

```

if (isspace(ch))
    continue;

if (isalnum(ch))
{
    postfix[k++] = ch;
}
else if (ch == '(')
{
    push(ch);
}
else if (ch == ')')
{
    while (top != -1 && peek() != '(')
        postfix[k++] = pop();
    pop(); // remove '('
}
else if (isOperator(ch))
{
    // FIX: handle right-associativity for '^'
    while (top != -1 && isOperator(peek()) &&
        ((precedence(peek()) > precedence(ch))
||
        (precedence(peek()) == precedence(ch)
&& ch != '^')))
        postfix[k++] = pop();

```

```

        push(ch);
    }
}

while (top != -1)
    postfix[k++] = pop();

postfix[k] = '\0';
}

// ----- Postfix Evaluation -----
double evaluatePostfix(char postfix[])
{
    int i;
    char ch;

    for (i = 0; postfix[i] != '\0'; i++)
    {
        ch = postfix[i];

        if (isspace(ch))
            continue;

        if (isalpha(ch))
        {
            double val;

            printf("Enter value of %c: ", ch);

```

```

        scanf("%lf", &val);
        pushEval(val);
    }
else if (isdigit(ch))
{
    pushEval(ch - '0');
}
else if (isOperator(ch))
{
    double b = popEval();
    double a = popEval();
    double res = 0;

    switch (ch)
    {
    case '+':
        res = a + b;
        break;
    case '-':
        res = a - b;
        break;
    case '*':
        res = a * b;
        break;
    case '/':
        if (b == 0)
        {

```

```

        printf("Error: Division by zero!\n");
        return 0;
    }
    res = a / b;
    break;
case '^':
    res = pow(a, b);
    break;
default:
    res = 0;
}

pushEval(res);
}

return popEval();
}

// ----- MAIN MENU -----

int main()
{
    char infix[MAX], postfix[MAX];
    int choice;
    double result;

    infix[0] = '\0';

```

```

while (1)
{
    printf("\n===== MENU =====\n");
    printf("1. Input Infix Expression\n");
    printf("2. Convert to Postfix\n");
    printf("3. Evaluate Postfix\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    getchar(); // clear newline

    switch (choice)
    {
    case 1:
        printf("Enter infix expression: ");
        fgets(infix, MAX, stdin);
        infix[strcspn(infix, "\n")] = '\0';
        break;

    case 2:
        if (strlen(infix) == 0)
        {
            printf("Please input an infix expression
first!\n");
        }
        else
        {

```

```

        top = -1;
        infixToPostfix(infix, postfix);
        printf("Postfix Expression: %s\n",
postfix);
    }
    break;

case 3:
    if (strlen(infix) == 0)
    {
        printf("Please input infix first!\n");
    }
    else
    {
        top = -1;
        evalTop = -1;
        infixToPostfix(infix, postfix);
        printf("Postfix Expression: %s\n",
postfix);

        result = evaluatePostfix(postfix);
        printf("Result of Evaluation: %.4lf\n",
result);
    }
    break;

case 4:
    printf("Exiting program...\n");
    exit(0);

```

```

        default:
            printf("Invalid choice! Try again.\n");
        }
    }

    return 0;
}

/*
    1.Right Associative operators in this
    ^      =      +=      -
=  *=  /=  %=  <<=  >>=  &=  ^=  |=  ?:  ++  --  (unary +,
-, !, ~)

*/

```

Output :

```
===== MENU =====
1. Input Infix Expression
2. Convert to Postfix
3. Evaluate Postfix
4. Exit
Enter your choice: 1
Enter infix expression: a^b^c
```

```
===== MENU =====
1. Input Infix Expression
2. Convert to Postfix
3. Evaluate Postfix
4. Exit
Enter your choice: 2
Postfix Expression: abc^^
```

```
===== MENU =====
1. Input Infix Expression
2. Convert to Postfix
3. Evaluate Postfix
4. Exit
Enter your choice: 3
Postfix Expression: abc^^
Enter value of a: 1
Enter value of b: 2
Enter value of c: 3
Result of Evaluation: 1.0000
```